

Back-end Frameworks

Sessão do usuário

Cadastro, Login e Recuperação de Senha

Parte 1

Profº: Joseph Donald

Contatos:

 www.josephdonald.wordpress.com

 @josephdonald

 030106382@prof.uninassau.edu.br

“O sucesso consiste em ir de fracasso em fracasso sem perder o entusiasmo.”
Winston Churchill



O que é Sessão de Usuário?

- ✓ A sessão de usuário é um mecanismo utilizado para manter o estado de autenticação e interação de um usuário durante sua navegação em qualquer aplicação.
- ✓ Como o protocolo HTTP é stateless (sem estado), o backend precisa de mecanismos que permitam identificar requisições pertencentes ao mesmo usuário.
- ✓ Segundo Roy Fielding (Criador das APIs/REST), a ausência de estado no HTTP melhora a escalabilidade das aplicações, porém exige estratégias complementares para gerenciamento de autenticação e persistência temporária de informações.



A sessão é responsável por:

- ✓ Manter usuários autenticados
 - ✓ Controlar permissões de acesso
 - ✓ Armazenar dados temporários
 - ✓ Garantir segurança durante a navegação
 - ✓ Melhorar a experiência do usuário
- De acordo com Martin Fowler (famoso autor da área de Arquitetura de Software), o gerenciamento de estado é essencial para sistemas corporativos, pois permite rastreabilidade, segurança e consistência nas operações realizadas pelos usuários.





O funcionamento básico ocorre em etapas:

1. Usuário realiza login
2. Backend valida credenciais
3. Servidor cria uma sessão
4. Um identificador é enviado ao cliente
5. O cliente envia esse identificador nas próximas requisições

Segundo Andrew S. Tanenbaum, sistemas distribuídos dependem fortemente de mecanismos de identificação e persistência temporária para manter comunicação confiável entre cliente e servidor.



As sessões contribuem diretamente para a segurança da aplicação:

- Evitam envio constante de senha
- Permitem expiração automática
- Reduzem riscos de acesso indevido
- Facilitam logout e invalidação de acessos

Conforme Bruce Schneier (especialista em segurança computacional), autenticação segura depende não apenas da validação inicial do usuário, mas também da proteção contínua da sessão durante toda a comunicação.



UNINASSAU

Sessão Tradicional vs JWT

Profº: Joseph Donald

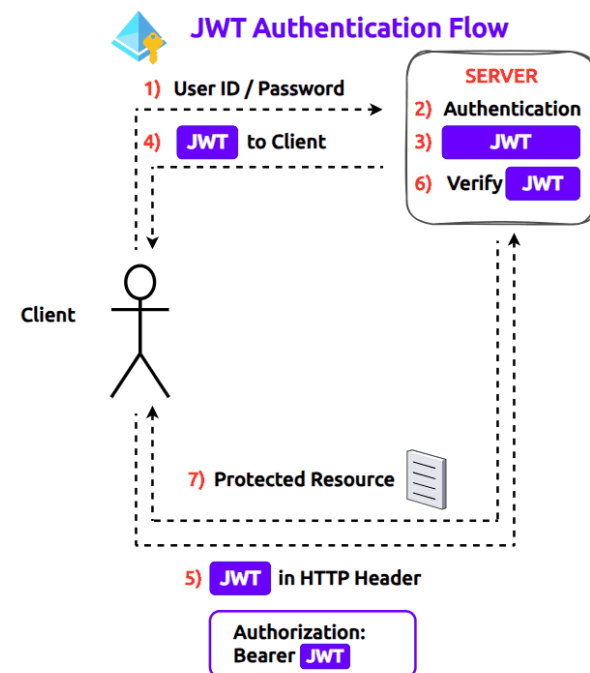
Sessão Tradicional

- Dados armazenados no servidor
- Mais controle centralizado
- Fácil invalidação

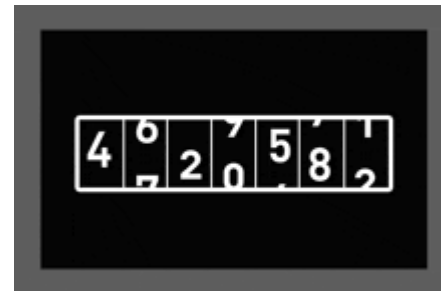
JWT (JSON Web Token)

- Dados armazenados no token
- Melhor escalabilidade
- Muito utilizado em APIs REST

Segundo Leonard Richardson, tokens JWT se tornaram populares por reduzirem dependência de armazenamento de sessão no servidor, favorecendo arquiteturas distribuídas.



- **Flask-Mail:** O Flask-Mail é uma extensão do Flask utilizada para integrar serviços de envio de e-mail em aplicações Python.
- **Random:** A biblioteca random do Python gera números pseudoaleatórios usando o algoritmo Mersenne Twister.





O que é o Flask-Mail

O Flask-Mail é uma extensão do Flask utilizada para integrar serviços de envio de e-mail em aplicações Python.

- ✓ Ele simplifica a comunicação SMTP, permitindo que o backend envie mensagens como:
- ✓ Recuperação de senha
- ✓ Confirmação de cadastro
- ✓ Notificações
- ✓ Alertas do sistema
- ✓ Verificação de conta

A biblioteca abstrai a complexidade do protocolo SMTP, facilitando a implementação de funcionalidades de comunicação em aplicações web.

Comando para a instalação do Flask-Mail:

pip install Flask-Mail



O funcionamento ocorre em etapas:

1. O backend configura um servidor SMTP
2. O Flask-Mail estabelece conexão com o servidor
3. Uma mensagem é criada
4. O servidor autentica o envio
5. O e-mail é enviado ao destinatário

Fluxo básico:

Usuário → Flask → Flask-Mail → SMTP → Destinatário

O Flask-Mail funciona como intermediador entre a aplicação Flask e o servidor de e-mail.



O Flask-Mail depende de um servidor SMTP para realizar o envio das mensagens.
Exemplo com Gmail:

```
app.config['MAIL_SERVER'] = 'smtp.gmail.com'  
app.config['MAIL_PORT'] = 587  
app.config['MAIL_USE_TLS'] = True  
app.config['MAIL_USERNAME'] = 'email@gmail.com'  
app.config['MAIL_PASSWORD'] = 'senha_app'
```

Principais configurações:

- MAIL_SERVER → servidor SMTP
- MAIL_PORT → porta de conexão
- MAIL_USE_TLS → criptografia TLS
- MAIL_USERNAME → conta remetente
- MAIL_PASSWORD → senha de autenticação



Após configurar o Flask-Mail, é criada uma instância da mensagem utilizando a classe Message.

Exemplo:

```
msg = Message(  
    subject='Recuperação de Senha',  
    sender='email@gmail.com',  
    recipients=['usuario@gmail.com']  
)  
  
msg.body = 'Seu código é 123456'
```

A mensagem pode conter:

- Assunto
- Destinatários
- Texto simples
- HTML
- Anexos



O envio é realizado utilizando:

```
mail.send(msg)
```

Nesse momento:

- O Flask-Mail conecta ao SMTP
- Realiza autenticação
- Transmite a mensagem
- O servidor encaminha ao destinatário

O backend pode utilizar isso para:

- Recuperação de senha
- Verificação de usuário
- Notificações automáticas
- Envio de relatórios

OBS: Antes de implementar o código, deve-se gerar a senha de app no seu Gmail.



Mudanças no Código

OBS: Como agora vamos trabalhar com sessões de usuário, os códigos das rotas sofreram uma alteração importante em relação aos códigos dos slides anteriores.

As rotas passaram a fazer o recebimento do “idUserarioFK”, pois quando o usuário fizer o login o sistema deverá trazer **somente** os dados do usuário logado.

Ex:

```
## CREATE - CADASTRO DO FUNCIONARIO
@app.route('/cadastraFuncionario', methods=['POST'])
def cadastraFuncionario():
    dadosRecebidos = request.get_json()

    nome = dadosRecebidos["nomeFuncionario"]
    telefone = dadosRecebidos["telefoneFuncionario"]
    idUsuarioFK = dadosRecebidos["idUserarioFK"]
    banco = conectaDB()
    cursor = banco.cursor()
```

A rota("/") foi transformada para '/lerFuncionario/<int:id_usuario>', pois agora ela carrega somente os funcionários cadastrados para aquele usuário.

```
@app.route('/lerFuncionario/<int:id_usuario>', methods=['GET'])
def lerFuncionario(id_usuario):
```



Primeiro passo:

- ✓ Configurar o **SMTP** do Gmail (tutorial disponível no próximo slide)

Segundo passo:

- ✓ Importar o novo banco de dados para o MySQL (arquivo no Teams)

Terceiro passo:

- ✓ Criar o projeto Python e ativar a venv

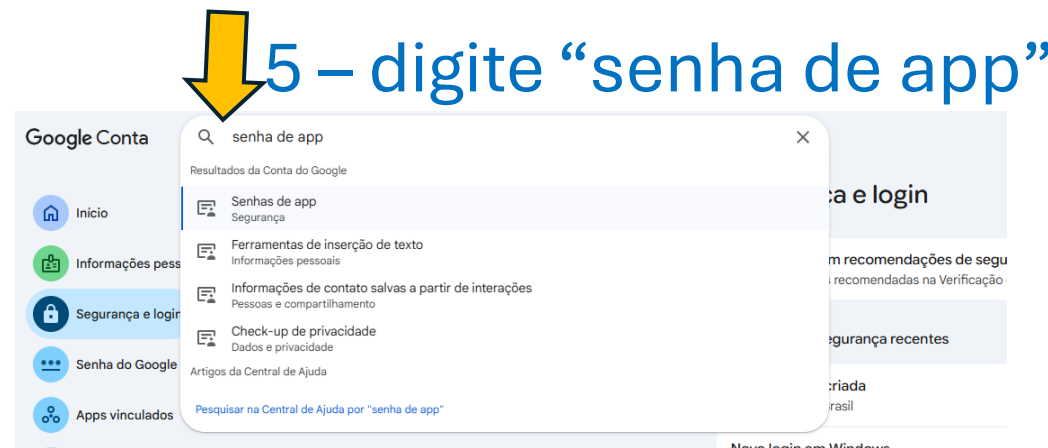
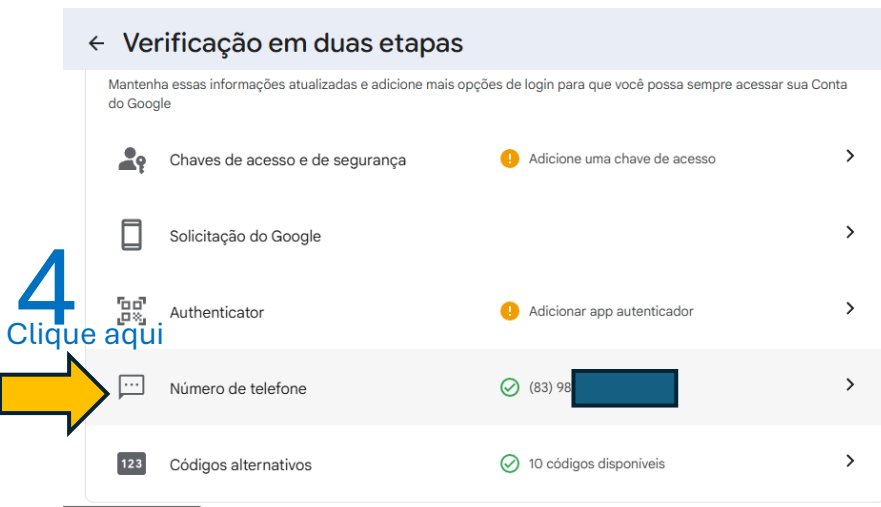
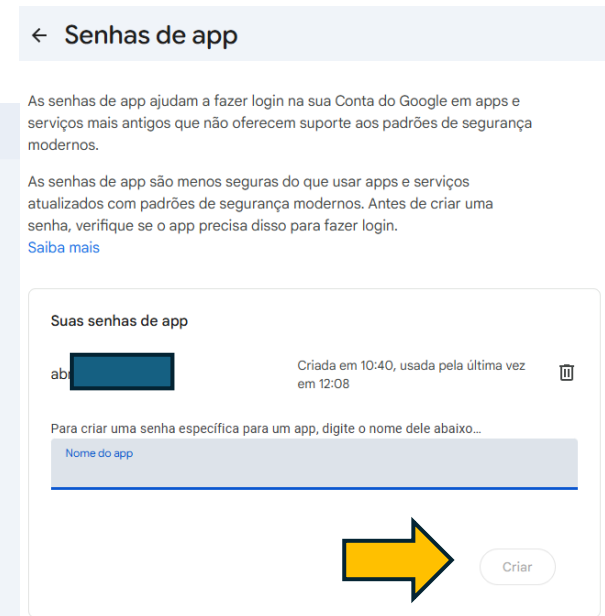
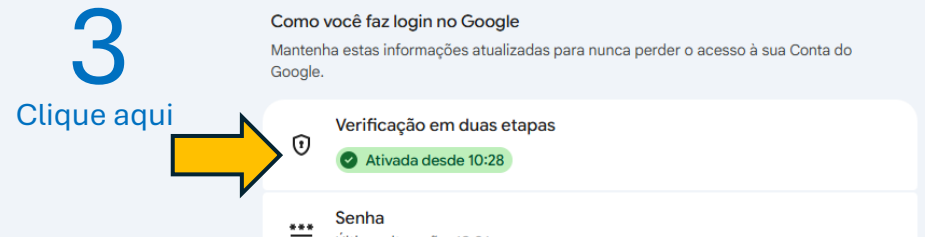
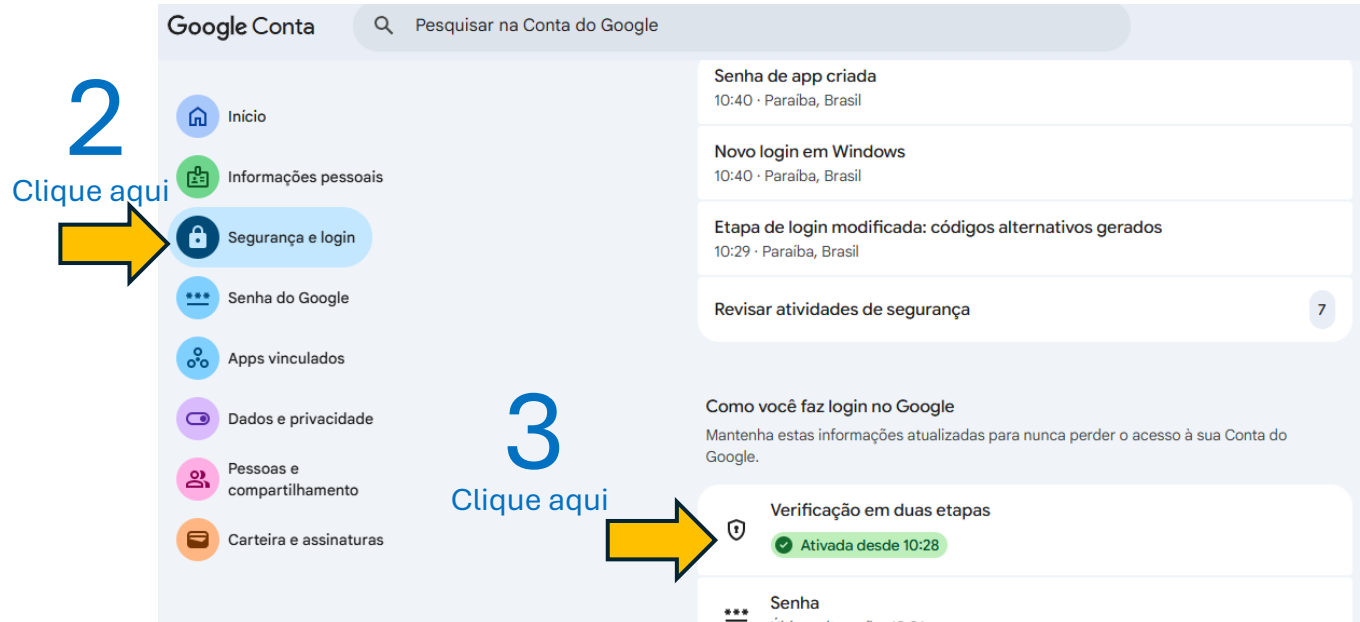
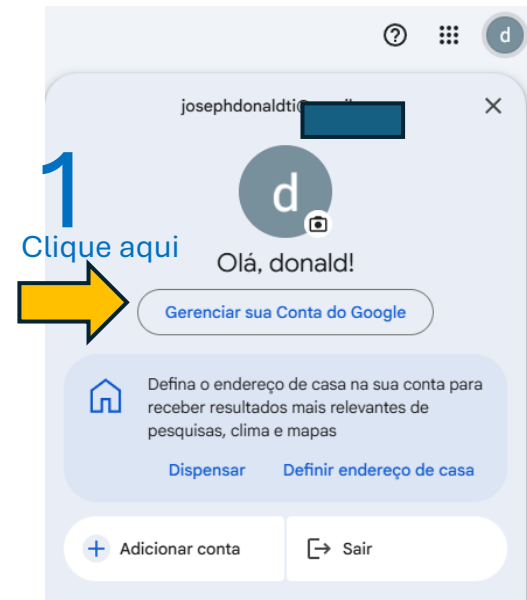
Ex: python -m venv venv

- ✓ Ativar a venv

venv\Scripts\activate.bat

Configuração do SMTP no GMAIL

Profº: Joseph Donald



7 - Exemplo da senha gerada

woob ktbk lspu vfbp



UNINASSAU

Código completo – parte 1

Profº: Joseph Donald

Quarto passo

Implemente o código

```
main.py > atualizaFuncionario
1 from flask import Flask, request, jsonify
2 import pymysql
3 from flask_mail import Mail, Message
4 import random
5
6 app = Flask(__name__)
7
8 def conectaDB():
9     db = pymysql.connect(
10         host='localhost',
11         database='banco_funcionarios',
12         user='root',
13         passwd=''
14     )
15
16     return db
17
18
19 ## CREATE - CADASTRO DO FUNCIONARIO
20 @app.route('/cadastraFuncionario', methods=['POST'])
21 def cadastraFuncionario():
22     dadosRecebidos = request.get_json()
23
24     nome = dadosRecebidos["nomeFuncionario"]
25     telefone = dadosRecebidos["telefoneFuncionario"]
26     idUsuarioFK = dadosRecebidos["idUsuarioFK"]
27     banco = conectaDB()
28     cursor = banco.cursor()
29
30     sql = f"INSERT INTO funcionario(nome, telefone, id_usuario_fk) VALUES ('{nome}', '{telefone}', {idUsuarioFK})"
31     try:
32         cursor.execute(sql)
33         banco.commit()
34         response = {"mensagem": "Cadastrado com sucesso", "codigo": 200}
35     except Exception as e:
36         response = {"mensagem": "Erro ao cadastrar funcionario", "codigo": 400, "erro": str(e)}
37     finally:
38         banco.close()
39
40     return jsonify(response)
41
```




```
42
43 # rota para atualização dos dados do funcionário nomeada como /atualizaFuncionario, utilizando o método PUT
44 @app.route('/atualizaFuncionario', methods=['PUT'])
45 def atualizaFuncionario():
46     dadosRecebidos = request.get_json()
47     idFuncionario = dadosRecebidos["idFuncionario"]
48     nomeFuncionario = dadosRecebidos["nomeFuncionario"]
49     telefone = dadosRecebidos["telefoneFuncionario"]
50     idUsuarioFK = dadosRecebidos["idUsuarioFK"]
51
52     banco = conectaDB()
53     cursor = banco.cursor()
54
55     sql = f"UPDATE funcionario SET nome = '{nomeFuncionario}', telefone = '{telefone}' WHERE id_funcionario = {idFuncionario} AND id_usuario_fk = {idUsuarioFK}"
56     try:
57         cursor.execute(sql)
58         banco.commit()
59         response = {"mensagem": "Atualizado com sucesso", "codigo": 200}
60     except Exception as e:
61         response = {"mensagem": "Erro ao atualizar funcionário", "codigo": 400, "erro": str(e)}
62     finally:
63         banco.close()
64
65     return jsonify(response)
66
67
68 ## READ - LER OS FUNCIONARIOS DO BANCO somente do usuário logado, utilizando o método GET e recebendo o id do usuário como parâmetro na URL
69 @app.route('/lerFuncionario/<int:id_usuario>', methods=['GET'])
70 def lerFuncionario(id_usuario):
71     banco = conectaDB()
72     cursor = banco.cursor()
73
74     sql = f"SELECT * FROM funcionario WHERE id_usuario_fk = {id_usuario};"
75     try:
76         cursor.execute(sql)
77         resultado = cursor.fetchall()
78         response = {"mensagem": "Funcionários lidos com sucesso", "codigo": 200}
79     except Exception as e:
80         response = {"mensagem": "Erro ao ler funcionarios", "codigo": 400, "erro": str(e)}
81     finally:
82         banco.close()
83
84     listaFuncionario = []
85
86     for funcionario in resultado:
87         listaFuncionario.append(
88             {
89                 "id": funcionario[0],
90                 "nomeFuncionario": funcionario[1],
91                 "telefoneFuncionario": funcionario[2],
92                 "idUsuarioFK": funcionario[3]
93             }
94         )
95
96     return jsonify(listaFuncionario, response)
97
```



```
97
98 # Rota para deletar um funcionário utilizando o método DELETE
99 @app.route('/deletaFuncionario', methods=['DELETE'])
100 def deletaFuncionario():
101     dadosRecebidos = request.get_json()
102     idFuncionario = dadosRecebidos["idFuncionario"]
103     idUsuarioFK = dadosRecebidos["idUsuarioFK"]
104
105     banco = conectaDB()
106     cursor = banco.cursor()
107
108     sql = f"DELETE FROM funcionario WHERE id_funcionario = {idFuncionario} AND id_usuario_fk = {idUsuarioFK}"
109     try:
110         cursor.execute(sql)
111         banco.commit()
112         response = {"mensagem": "Deletado com sucesso", "codigo": 200}
113     except Exception as e:
114         response = {"mensagem": "Erro ao deletar funcionário", "codigo": 400, "erro": str(e)}
115     finally:
116         banco.close()
117
118     return jsonify(response)
119
120
121 ## CREATE - ROTA PARA CADASTRAR UM USUÁRIO
122 @app.route('/cadastraUsuario', methods=['POST'])
123 def cadastraUsuario():
124     dadosRecebidos = request.get_json()
125     nome = dadosRecebidos["nomeUsuario"]
126     email = dadosRecebidos["emailUsuario"]
127     senha = dadosRecebidos["senhaUsuario"]
128
129     banco = conectaDB()
130     cursor = banco.cursor()
131
132     sql = f"INSERT INTO usuario(nome, email, senha) VALUES ('{nome}', '{email}', '{senha}')"
133     try:
134         cursor.execute(sql)
135         banco.commit()
136         response = {"mensagem": "Cadastrado com sucesso", "codigo": 200}
137     except pymysql.err.IntegrityError as e:
138         response = {"mensagem": "Email já cadastrado", "codigo": 400, "erro": str(e)}
139     finally:
140         banco.close()
141
142     return jsonify(response)
143
```



```
145  ## LOGIN - ROTA PARA REALIZAR O LOGIN DO USUÁRIO recolhendo todas as informações do usuário para realizar a autenticação
146  @app.route('/loginUsuario', methods=['POST'])
147  def loginUsuario():
148      dadosRecebidos = request.get_json()
149      email = dadosRecebidos["emailUsuario"]
150      senha = dadosRecebidos["senhaUsuario"]
151
152      banco = conectaDB()
153      cursor = banco.cursor()
154
155      sql = f"SELECT id_usuario, nome, email, senha FROM usuario WHERE email = '{email}' AND senha = '{senha}'"
156      try:
157          cursor.execute(sql)
158          resultado = cursor.fetchone()
159          response = {"mensagem" : "Login realizado com sucesso", "codigo" : 200, "usuario" : {
160              "id" : resultado[0],
161              "nome" : resultado[1],
162              "email" : resultado[2]
163          }}
164      except Exception as e:
165          response = {"mensagem" : "E-mail ou senha incorretos", "codigo" : 401, "erro" : str(e)}
166      finally:
167          banco.close()
168
169      return jsonify(response)
170
```



```
171
172 ## rota de recuperação de senha através do Flask-Mail, utilizando o método POST para enviar um e-mail de recuperação com um código temporário de confirmação
173 @app.route('/recuperaSenha', methods=['POST'])
174 def recuperaSenha():
175
176     global codigo_recuperacao_gerado
177
178     dadosRecebidos = request.get_json()
179     email = dadosRecebidos["emailUsuario"]
180
181     banco = conectaDB()
182     cursor = banco.cursor()
183
184     sql = f"SELECT id_usuario, nome, email FROM usuario WHERE email = '{email}'"
185     try:
186         cursor.execute(sql)
187         resultado = cursor.fetchone()
188         if resultado:
189
190             app.config['MAIL_SERVER'] = 'smtp.gmail.com'
191             app.config['MAIL_PORT'] = 587
192             app.config['MAIL_USE_TLS'] = True
193             app.config['MAIL_USERNAME'] = 'josephdonald@gmail.com'
194             app.config['MAIL_PASSWORD'] = 'coyq fwjt wfbl kxtv'
195             mail = Mail(app)
196
197             msg = Message("Recuperação de Senha", sender="josephdonald@gmail.com", recipients=[email])
198             codigo_recuperacao_gerado = random.randint(100000, 999999)
199             msg.body = f"Olá seu código de recuperação de senha é: {codigo_recuperacao_gerado}"
200
201             mail.send(msg)
202             response = {"mensagem" : "E-mail de recuperação enviado com sucesso", "codigo" : 200, "usuario" : {
203                 "id" : resultado[0],
204                 "nome" : resultado[1],
205                 "email" : resultado[2]
206             }}
207         else:
208             response = {"mensagem" : "E-mail não encontrado", "codigo" : 404}
209     except Exception as e:
210         response = {"mensagem" : "Erro ao recuperar senha", "codigo" : 400, "erro" : str(e)}
211     finally:
212         banco.close()
213
214     return jsonify(response)
215
```



```
215
216 # Rota para atualizar a senha do usuário utilizando o método PUT, recebendo o email do usuário e a nova senha
217 @app.route('/atualizaSenha', methods=['PUT'])
218 def atualizaSenha():
219     dadosRecebidos = request.get_json()
220     email = dadosRecebidos["emailUsuario"]
221     nova_senha = dadosRecebidos["novaSenha"]
222     codigo_recuperacao_recebido = dadosRecebidos["codigoRecuperacao"]
223
224     print("Código de recuperação recebido:", codigo_recuperacao_recebido)
225     print("Código de recuperação gerado:", codigo_recuperacao_gerado)
226
227     if codigo_recuperacao_recebido != codigo_recuperacao_gerado:
228         return jsonify({"mensagem": "Código de recuperação inválido", "codigo": 400})
229
230     banco = conectaDB()
231     cursor = banco.cursor()
232
233     sql = f"UPDATE usuario SET senha = '{nova_senha}' WHERE email = '{email}';"
234     try:
235         cursor.execute(sql)
236         banco.commit()
237         response = {"mensagem": "Senha atualizada com sucesso", "codigo": 200}
238     except Exception as e:
239         response = {"mensagem": "Erro ao atualizar senha", "codigo": 400, "erro": str(e)}
240     finally:
241         banco.close()
242
243     return jsonify(response)
244
245 if __name__ == "__main__":
246     app.run(debug=True)
247
248
```